프로젝트 기획서 1장 (템플릿 01)

머신러닝 프로젝트(RISE) · 개인 프로젝트 — 「줴줴이야」 리그오브레전드 승패 예측 시뮬레이션 & 챗 게임

항목	내용
이름 / 아이디	박대승 · 학번 `22017041` · 서버 아이디 `262mb22017041` · 프로젝트 폴더 `~/work/week4/gg_battle`
서비스 한 줄	**게임 리그오브레전드에서 게임의 요소 지표를 '감'으로 판단하는 유저를 위해, 각 행동이 승률을 얼마나 바꾸는지 9,879경기의 10분(당시) 데이터를 모델로 분해해 보여주고, 시뮬레이션에서 마치는 것이 아닌 실제 1대1 대결로 채팅 게임을 지원하는 서비스**
대상 사용자 · 문제	**대상** — 롤 초반 지표(골드·킬·CS·오브젝트)를 읽을 줄 아는 플레이어. 넓게는 **승률 모델의 설명을 납득해야 하는 사람** 전부. **지금은**: 전적 사이트는 **끝난 경기의 결과값**만 보여주고, "드래곤이 중요하다"는 말은 많아도 **몇 %p인지 수치가 없다.** 직접 모델을 돌려도 결과는 **계수표 한 장**이라 체감되지 않는다. 특히 **골드를 통제하면 타워·킬·CS 계수가 음수로 뒤집히는데**, 글로 읽어서는 납득되지 않고 보통 "버그"로 치부된다. **이 서비스가 하는 것**: ① **선형 모델**이라 승률을 항별로 정확히 쪼개 *무엇이 몇 %p*로 답한다(SHAP 불필요) ② 반직관적 계수를 숨기지 않고 **단독 상관 ↔ 골드 통제 비교**로 원본 9,879경기에서 증명한다 ③ 계수를 **읽는** 대신 60초 동안 **직접 쳐서** 체감한다 — 쿨타임·스폰·캡까지 데이터에서 환산했으므로 **효율 높은 행동을 고르는 것이 곧 전략**이 된다.
핵심 기능 3개 (Must)	**① 승률 분해** — "10분 지표 차이를 넣으면 승률 %와 **어느 항이 몇 %p 기여했는지** 보고 싶다" → `core.win_rate()` + 상태 슬라이더 *(1순위 · 완료)* **② 데이터 근거 확인** — "`towerDiff` 계수가 **왜 음수인지** 원본 9,879경기에서 직접 확인하고 싶다" → 단독 상관 ↔ 골드 통제 비교 화면 *(2순위 · 완료)* **③ 60초 1대1 대결** — "친구와 같은 방에 들어가 **채팅 명령**으로 승률을 끌어당기며 겨루고 싶다" → 양쪽 큐를 한 시간축에서 돌리는 대결 화면 *(3순위 · 완료)*
화면 목록 · 데이터	**4화면**: ① 홈(규칙·명령어 환율표) ② 데이터 탐색 ③ 시뮬레이터 ④ 1대1 대결 **데이터 확보 완료** — `data/high_diamond_ranked_10min.csv` · **9,879행 × 40열 · 결측 0건 · `blueWins` 49.9%**(균형) 출처: Kaggle `bobbyscience/league-of-legends-diamond-ranked-games-10-min` (2020) · 로그인 없이 받는 미러 `github.com/jppoter/Project-on-high_diamond_ranked_10min-dataset` · 정리된 열 목록은 `data/README.md`
2·3단계 연결	**입력** 11개 차이 피처(골드·경험·킬·CS·정글·드래곤·전령·타워·와드·와드제거·퍼블) → **예측값** 승률 `{"score": int, "level": str, "reasons": list[str]}` 1단계 로지스틱 회귀 **CV AUC 0.8106** → 2단계 트리 계열과 비교 교체 → 3단계 FastAPI `POST /predict`. **반환 형식을 지금 고정했으므로 화면 코드는 한 줄도 안 고친다.**
리스크 1개와 대안	2기기 동시 대결이 JupyterHub 프록시 제약으로 **같은 계정으로 로그인한 브라우저**만 가능하다(발표자 노트북 + 발표자 휴대폰). → 시연 당일 1대1 실제 플레이가 안 되면 **시뮬레이터 단독 모드 + `bot_test.py` 봇 대결**로 동일 로직을 시연한다.

주제 조건 4개 — 통과 확인

조건	상태
① 5주차에 시연 가능	**통과** · Must 3개 구현 완료 · 4화면 동작 · 외부 계정·승인 불필요
② 데이터가 있다	**통과** · 공개 데이터를 `data/`에 실제 파일로 확보 (9,879 × 40)
③ 예측 기능이 붙는다	**통과** · 11개 차이 피처 → 승률(%) · CV AUC 0.8106
④ 사용자가 구체적	**통과** · 게임 리그오브레전드에서 게임의 요소 지표를 '감'으로 판단하는 유저

현재 검증 현황 · `python -m pytest -q` **50 passed** (Logic Validation) · `bot_test.py` 타자 속도 9케이스 × 전략 10종 밸런스 검증 · `browser_test.py` 2세션 동시 대결·모바일 렌더 확인. **밸런스 수치(계수·골드·경험·캡·쿨타임)를 전부 데이터에서 유도했고, 설계로 정한 것은 `N`타와 `SEC`(타당 처리시간) 둘뿐이다.**